

COURSE CODE: 17YCM603

COURSE NAME: SEARCH ALGORITHMS

UNIT-4 SYLLABUS

LECTURE: 3HRS/WEEK

MARKS: 100

OBJECTIVES:

To Learn the basics of search algorithms.
To understand and apply various types of search algorithms.
To understand the concept of Heuristic Search.
To apply different types of Hill Climbing Algorithms.
To learn the various crypto currency and bitcoin algorithm

OUTCOMES

At the end of the course, the learners will be able to:	
1	Implement search algorithms in real world applications.
2	Apply heuristic search as a searching technique.
3	Apply hill climbing algorithm to solve the problems
4	Explain and implement BFS and DFS algorithm
5	Discuss cryptocurrency, bitcoin and its algorithm.

TEXTBOOKS

Text Books
1. Thomas H. Cormen , Charles E. Leiserson , Ronald L. Rivest , Clifford Stein, “Introduction to Algorithms, 3rd Edition, The MIT Press, 3rd Edition, 2009.
2. Kosko, B, “Neural Networks and FuzzySystems: A Dynamical Approach to Machine Intelligence”, PrenticeHall, NewDelhi, 2004.
Reference Books
1.Jack M. Zurada, “Introduction to Artificial Neural Systems”, PWS Publishing Co., Boston, 2002.
2. Klir G.J. & Folger T.A., “Fuzzysets, Uncertainty and Information”, Prentice–Hall of India Pvt. Ltd., New Delhi, 2008

SYLLABUS:

Module 1: Hill Climbing, Hill Climbing Algorithm, Steepest Ascent Hill Climbing, Stochastic Hill Climbing, Disadvantages of Hill Climbing, Foothills, Plateaus and Ridges

Module 2: Introduction of Crypto currency and bit coin, Applications of Crypto currency and Bit coin, SHA-2 Bit coin Algorithm, Scrypt: The Most complete Cryptographic function, X11, Dash Algorithm.

MODULE 1:

HILL CLIMBING

HILL CLIMBING

Hill Climbing is a heuristic search algorithm used primarily for mathematical optimization problems in artificial intelligence (AI). It is a form of local search, which means it focuses on finding the optimal solution by making incremental changes to an existing solution and then evaluating whether the new solution is better than the current one. The process is analogous to climbing a hill where you continually seek to improve your position until you reach the top, or local maximum, from where no further improvement can be made.

Hill climbing is a fundamental concept in AI because of its simplicity, efficiency, and effectiveness in certain scenarios, especially when dealing with optimization problems or finding solutions in large search spaces.

How Does the Hill Climbing Algorithm Work?

In the **Hill Climbing algorithm**, the process begins with an initial solution, which is then iteratively improved by making small, incremental changes. These changes are evaluated by a **heuristic function** to determine the quality of the solution. The algorithm continues to make these adjustments until it reaches a **local maximum**—a point where no further improvement can be made with the current set of moves.

Basic Concepts of Hill Climbing Algorithms

Hill climbing follows these steps:

1. **Initial State:** Start with an arbitrary or random solution (initial state).
2. **Neighboring States:** Identify neighboring states of the current solution by making small adjustments (mutations or tweaks).
3. **Move to Neighbor:** If one of the neighboring states offers a better solution (according to some evaluation function), move to this new state.
4. **Termination:** Repeat this process until no neighboring state is better than the current one. At this point, you've reached a local maximum or minimum (depending on whether you're maximizing or minimizing).

Hill Climbing as a Heuristic Search in Mathematical Optimization

Hill Climbing algorithm often used for solving mathematical optimization problems in AI. With a good heuristic function and a large set of inputs, Hill Climbing can find a sufficiently good solution in a reasonable amount of time, although it may not always find the global optimal maximum. In mathematical optimization, Hill Climbing is commonly applied to problems that involve maximizing or minimizing a real function. For example, in

the Traveling Salesman Problem, the objective is to minimize the distance traveled by the salesman while visiting multiple cities.

What is a Heuristic Function?

A **heuristic function** is a function that ranks the possible alternatives at any branching step in a search algorithm based on available information. It helps the algorithm select the best route among various possible paths, thus guiding the search towards a good solution efficiently.

Features of the Hill Climbing Algorithm

1. **Variant of Generating and Testing Algorithm:** Hill Climbing is a specific variant of the generating and testing algorithms. The process involves:
 - **Generating possible solutions:** The algorithm creates potential solutions within the search space.
 - **Testing solutions:** Each generated solution is evaluated to determine if it meets the desired criteria.
 - **Iteration:** If a satisfactory solution is found, the algorithm terminates; otherwise, it returns to the generation step.This iterative feedback mechanism allows Hill Climbing to refine its search by using information from previous evaluations to inform future moves in the search space.
2. **Greedy Approach:** The Hill Climbing algorithm employs a greedy approach, meaning that at each step, it moves in the direction that optimizes the objective function. This strategy aims to find the optimal solution efficiently by making the best immediate choice without considering the overall problem context.

Types of Hill Climbing in Artificial Intelligence

1. Simple Hill Climbing Algorithm

Simple Hill Climbing is a straightforward variant of hill climbing where the algorithm evaluates each neighboring node one by one and selects the first node that offers an improvement over the current one.

Algorithm for Simple Hill Climbing

1. Evaluate the initial state. If it is a goal state, return success.
2. Make the initial state the current state.
3. Loop until a solution is found or no operators can be applied:
 - Select a new state that has not yet been applied to the current state.
 - Evaluate the new state.
 - If the new state is the goal, return success.
 - If the new state improves upon the current state, make it the current state and continue.
 - If it doesn't improve, continue searching neighboring states.
4. Exit the function if no better state is found.

2. Steepest-Ascent Hill Climbing

Steepest-Ascent Hill Climbing is an enhanced version of simple hill climbing. Instead of moving to the first neighboring node that improves the state, it evaluates all neighbors and moves to the one offering the highest improvement (steepest ascent).

Algorithm for Steepest-Ascent Hill Climbing

1. Evaluate the initial state. If it is a goal state, return success.
2. Make the initial state the current state.
3. Repeat until the solution is found or the current state remains unchanged:
 - Select a new state that hasn't been applied to the current state.
 - Initialize a 'best state' variable and evaluate all neighboring states.
 - If a better state is found, update the best state.
 - If the best state is the goal, return success.
 - If the best state improves upon the current state, make it the new current state and repeat.
4. Exit the function if no better state is found.

3. Stochastic Hill Climbing

Stochastic Hill Climbing introduces randomness into the search process. Instead of evaluating all neighbors or selecting the first improvement, it selects a random neighboring node and decides whether to move based on its improvement over the current state.

Algorithm for Stochastic Hill Climbing:

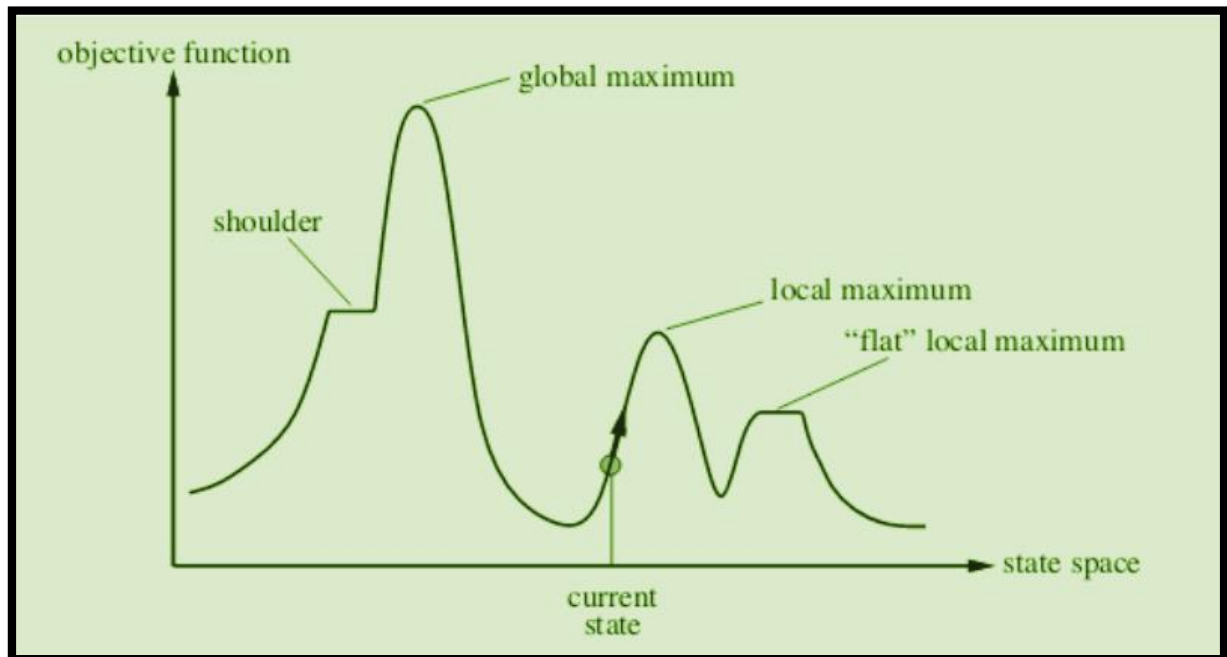
1. Evaluate the initial state. If it is a goal state, return success.
2. Make the initial state the current state.
3. Repeat until a solution is found or the current state does not change:
 - Apply the successor function to the current state and generate all neighboring states.
 - Choose a random neighboring state based on a probability function.
 - If the chosen state is better than the current state, make it the new current state.
 - If the selected neighbor is the goal state, return success.
4. Exit the function if no better state is found.

State-Space Diagram in Hill Climbing: Key Concepts and Regions

In the Hill Climbing algorithm, the state-space diagram is a visual representation of all possible states the search algorithm can reach, plotted against the values of the objective function (the function we aim to maximize). In the state-space diagram:

- X-axis: Represents the state space, which includes all the possible states or configurations that the algorithm can reach.
- Y-axis: Represents the values of the objective function corresponding to each state.

The optimal solution in the state-space diagram is represented by the state where the objective function reaches its maximum value, also known as the global maximum.



Key Regions in the State-Space Diagram

1. **Local Maximum:** A local maximum is a state better than its neighbors but not the best overall. While its objective function value is higher than nearby states, a global maximum may still exist.
2. **Global Maximum:** The global maximum is the best state in the state-space diagram, where the objective function achieves its highest value. This is the optimal solution the algorithm seeks.
3. **Plateau/Flat Local Maximum:** A plateau is a flat region where neighboring states have the same objective function value, making it difficult for the algorithm to decide on the best direction to move.
4. **Ridge:** A ridge is a higher region with a slope, which can look like a peak. This may cause the algorithm to stop prematurely, missing better solutions nearby.
5. **Current State:** The current state refers to the algorithm's position in the state-space diagram during its search for the optimal solution.
6. **Shoulder:** A shoulder is a plateau with an uphill edge, allowing the algorithm to move toward better solutions if it continues searching beyond the plateau.

Advantages of Hill Climbing Algorithm

1. **Simplicity and Ease of Implementation:** Hill Climbing is a simple and intuitive algorithm that is easy to understand and implement, making it accessible for developers and researchers alike.

2. **Versatility:** The algorithm can be applied to a wide variety of optimization problems, including those with large search spaces and complex constraints. It's especially useful in areas such as resource allocation, scheduling, and route planning.
3. **Efficiency in Finding Local Optima:** Hill Climbing is often highly efficient at finding local optima, making it a suitable choice for problems where a good solution is required quickly.
4. **Customizability:** The algorithm can be easily modified or extended to incorporate additional heuristics or constraints, allowing for more tailored optimization approaches.

Applications of Hill Climbing in AI

1. **Pathfinding:** Hill climbing is used in AI systems that need to navigate or find the shortest path between points, such as in robotics or game development.
2. **Optimization:** Hill climbing can be used for solving optimization problems where the goal is to maximize or minimize a particular objective function, such as scheduling or resource allocation problems.
3. **Game AI:** In certain games, AI uses hill climbing to evaluate and improve its position relative to an opponent's.
4. **Machine Learning:** Hill climbing is sometimes used for hyperparameter tuning, where the algorithm iterates over different sets of hyperparameters to find the best configuration for a machine learning model.

STEEPEST ASCENT HILL CLIMBING

Steepest ascent hill climbing is a variant of the basic hill climbing algorithm used in optimization problems. It's a local search algorithm that explores all neighboring states and selects the one that provides the highest increase in the objective function (or steepest descent if minimizing).

How It Works

Evaluate current state: Calculate the value of the current solution

Generate all neighbors: Create all possible neighboring states

Evaluate all neighbors: Calculate the objective function for each neighbor

Select best neighbor: Choose the neighbor with the highest improvement

Move to best neighbor: If better than current state, move there and repeat

Terminate: If no better neighbors exist, return current state as local optimum

Advantages over Basic Hill Climbing

- More thorough search than simple hill climbing (which just takes the first improving move)
- Less likely to get stuck on small local optima
- More systematic exploration of neighborhood

Disadvantages

- Computationally more expensive (must evaluate all neighbors)
- Still prone to getting stuck at local optima (like all hill climbing variants)
- May be slow in high-dimensional spaces with many neighbors

Applications

Steepest ascent hill climbing is used in various optimization problems where:

- The search space is too large for exhaustive search
- An approximate solution is acceptable
- The objective function is relatively smooth with some gradient to follow

STOCHASTIC HILL CLIMBING

Stochastic Hill Climbing is a variation of the classic Hill Climbing algorithm that introduces randomness to escape local optima and improve exploration. Unlike Steepest Ascent Hill Climbing, which always selects the best neighboring state, Stochastic Hill Climbing randomly selects among improving moves, making it less greedy and more flexible.

How Stochastic Hill Climbing Works

1. Start with an initial solution (current state).
2. Generate neighboring solutions (possible next states).
3. Evaluate each neighbor based on an objective (fitness) function.
4. Randomly select an uphill move (a neighbour better than the current state).
5. Move to the selected neighbor (even if it's not the steepest ascent).

6. Repeat until no better neighbors exist (local optimum reached).

Key Difference from Standard Hill Climbing

- **Basic Hill Climbing:** Takes the first improving move (greedy but fast).
- **Steepest Ascent Hill Climbing:** Takes the best possible move (computationally expensive).
- **Stochastic Hill Climbing:** Randomly selects any improving move (balances exploration & exploitation).

Advantages of Stochastic Hill Climbing

1. Less likely to get stuck in local optima compared to greedy methods. More exploration due to randomness.
2. Computationally cheaper than Steepest Ascent (doesn't evaluate all neighbors).
3. Good for large search spaces where exhaustive search is impractical.

Disadvantages

1. Not guaranteed to find the global optimum (still a local search method).
2. May converge slower than Steepest Ascent due to random selection.
3. Performance depends on randomness (may sometimes make poor choices).

Applications

1. Optimization problems (scheduling, logistics, parameter tuning).
2. Machine Learning (hyperparameter optimization, neural network training).
3. Game AI (heuristic search in strategy games).
4. Robotics (path planning with probabilistic improvements).

MODULE 2:

CRYPTOCURRENCY AND BITCOIN

INTRODUCTION TO CRYPTOCURRENCY AND BITCOIN

Cryptocurrency is a digital or virtual currency secured by cryptography, operating on decentralized networks (usually blockchains). **Bitcoin (BTC)**, created in 2009 by the pseudonymous **Satoshi Nakamoto**, is the first and most well-known cryptocurrency.

What is Cryptocurrency?

A cryptocurrency is:

- **Decentralized:** No central authority (like a bank or government) controls it.
- **Digital:** Exists only in electronic form, stored in cryptographic wallets.
- **Secure:** Uses cryptography for transaction verification and security.
- **Immutable:** Transactions, once confirmed, cannot be altered.
- **Transparent:** Most blockchains (like Bitcoin's) are public ledgers.

Types of Cryptocurrencies

- **Bitcoin (BTC):** The first cryptocurrency, primarily used as digital gold/store of value.
 - **Altcoins:** Alternative coins like Ethereum (ETH), Litecoin (LTC), Ripple (XRP).
 - **Stablecoins:** Pegged to fiat (e.g., USDT, USDC) to reduce volatility.
 - **Privacy Coins:** Monero (XMR), Zcash (ZEC) – focus on anonymity.
 - **Smart Contract Platforms:** Ethereum, Solana, Cardano – enable decentralized apps (DApps).
-

Key Features of Bitcoin

Limited Supply: Only 21 million BTC will ever exist (deflationary).

Decentralized: Run by a global network of nodes (no single point of failure).

Proof-of-Work (PoW): Miners secure the network by solving cryptographic puzzles.

Pseudonymous: Transactions are public, but identities are hidden behind

addresses.

Censorship-resistant: No government or bank can freeze Bitcoin transactions.

How Bitcoin Works

1. **Transactions:** Users send BTC to each other via cryptographic signatures.
2. **Mining:** Miners validate transactions and add them to the blockchain.
3. **Consensus:** Nodes enforce rules (e.g., no double-spending).
4. **Wallets:** Users store private keys to access their BTC (e.g., Ledger, Trezor, MetaMask).

Bitcoin's Use Cases

- **Digital Gold:** Store of value (like gold, but digital).
- **Peer-to-Peer Cash:** Send money globally without banks.
- **Inflation Hedge:** Limited supply protects against currency devaluation.
- **Institutional Asset:** Companies like Tesla & MicroStrategy hold BTC.

Blockchain: The Technology Behind Bitcoin

A blockchain is a public, immutable ledger that records all transactions.

- **Blocks:** Contain batches of transactions (Bitcoin adds a new block every ~10 minutes).
- **Decentralization:** No single entity controls the network.
- **Security:** Cryptographic hashing (SHA-256) and consensus mechanisms (PoW) prevent fraud.

Bitcoin vs. Traditional Finance

Feature	Bitcoin (BTC)	Traditional Banking
Control	Decentralized	Centralized (Banks/Govt)
Transaction Speed	~10 min (varies)	Seconds (but settlements take days)
Fees	Variable (depends on network congestion)	High (especially for cross-border)
Censorship	Resistant	Can freeze accounts
Supply	Fixed (21M BTC)	Unlimited (inflation possible)

Applications of Bitcoin & Cryptocurrency

Bitcoin and other cryptocurrencies have evolved beyond just digital money—they now power decentralized finance (DeFi), smart contracts, NFTs, and more. Below are the key real-world applications:

1. Digital Payments & Remittances

Fast & Borderless Transactions

- Send money globally in minutes (vs. days with banks).
- Example: A worker in the U.S. can send Bitcoin to family in Venezuela instantly.

Lower Fees Than Traditional Banking

- Avoid high wire transfer fees (especially for cross-border payments).
- Example: Using Bitcoin Lightning Network for near-zero-fee micropayments.

Censorship-Resistant Money

- Useful in countries with strict capital controls (e.g., Nigeria, Argentina).

2. Store of Value (Digital Gold)

Hedge against Inflation

- Bitcoin's fixed supply (21M BTC) protects against currency devaluation.
- Example: Countries like Turkey & Venezuela see high Bitcoin adoption due to hyperinflation.

Institutional Investment

- Companies like MicroStrategy, Tesla, & Square hold BTC as a treasury asset.
- Bitcoin ETFs (e.g., BlackRock's IBIT) allow traditional investors to gain exposure.

3. Decentralized Finance (DeFi)

DeFi replaces banks with blockchain-based financial services:

Lending & Borrowing

- Platforms like **Aave**, **Compound** let users earn interest on crypto deposits.
- No credit checks—loans are secured via crypto collateral.

Decentralized Exchanges (DEXs)

- Trade without intermediaries (e.g., **Uniswap, PancakeSwap**).

Stable coins for Payments

- **USDT, USDC** provide dollar-pegged stability for daily transactions.

4. Smart Contracts & dApps

(Ethereum, Solana, Cardano, etc.)

Automated Agreements

- Self-executing contracts (e.g., escrow, insurance payouts).

Supply Chain Tracking

- Walmart uses blockchain to trace food sources.

Decentralized Social Media

- Platforms like **Mastodon, Lens Protocol** resist censorship.

5. Non-Fungible Tokens (NFTs)

Digital Ownership & Art

- NFTs represent unique assets (e.g., CryptoPunks, Bored Apes).

Gaming & Virtual Assets

- Axie Infinity, Decentraland use NFTs for in-game items.

Real-World Asset Tokenization

- Houses, stocks, and even music rights can be tokenized (e.g., RealT for real estate).

6. Privacy & Anonymous Transactions

Financial Privacy

- Monero (XMR), Zcash (ZEC) hide transaction details.

Whistleblower Donations

- WikiLeaks accepts Bitcoin to avoid banking censorship.
-

7. Web3 & Decentralized Internet

User-Controlled Data

- No more Facebook/Google owning your data—blockchain enables self-sovereign identity.

Decentralized Storage

- **Filecoin, IPFS** replace cloud storage like AWS.
-

8. Bitcoin Mining & Energy Innovations

Renewable Energy Use

- Bitcoin miners use excess hydro, solar, and flare gas energy.

Grid Stabilization

- Miners act as flexible energy buyers, helping balance power demand.
-

9. Government & Enterprise Adoption

Central Bank Digital Currencies (CBDCs)

- China's digital yuan, Nigeria's eNaira are blockchain-based.

Corporate Payments

- Microsoft, AT&T, and Overstock accept Bitcoin.

Charity & Aid

- Ukraine received over \$100M in crypto donations during the war.

What is SHA-2?

- **SHA-2** is a family of cryptographic hash functions designed by the NSA and published in 2001.
- It includes different versions based on output size:

SHA-224 (224-bit hash)

SHA-256 (256-bit hash, used in Bitcoin)

SHA-384 (384-bit hash)

SHA-512 (512-bit hash)

2. SHA-256 in Bitcoin

Bitcoin relies heavily on **SHA-256** for:

Mining (Proof-of-Work)

- Miners compute **SHA-256 hashes** repeatedly to find a valid block hash.
- The mining difficulty adjusts based on the network's computational power.

Blockchain Security

- Each block contains the **SHA-256 hash of the previous block**, creating an immutable chain.
- Tampering with a block would require recalculating all subsequent hashes (computationally infeasible).

Transaction IDs (TXID)

- Every Bitcoin transaction is hashed using **SHA-256 + RIPEMD-160** to generate a unique identifier.

3. How SHA-256 Works

- Takes an input (any data) and produces a fixed 256-bit (32-byte) hash.
- Deterministic: Same input always produces the same output.
- Avalanche Effect: A tiny change in input completely changes the hash.
- Preimage Resistance: Extremely hard to reverse-engineer input from hash.

4. Why SHA-256 (Not SHA-II)?

- SHA-II is not a standard term—it likely refers to SHA-2.
- Bitcoin specifically uses SHA-256, not other SHA-2 variants (like SHA-512).
- SHA-256 was chosen for its balance of security, speed, and simplicity.

What is the SHA-256 Algorithm?

SHA 256 is a part of the SHA 2 family of algorithms, where SHA stands for Secure Hash Algorithm. Published in 2001, it was a joint effort between the NSA and NIST to introduce a successor to the SHA 1 family, which was slowly losing strength against brute force attacks. The significance of the 256 in the name stands for the final hash digest value, i.e. irrespective of the size of plaintext/cleartext, the hash value will always be 256 bits.

The other algorithms in the SHA family are more or less similar to SHA 256. Now, look into knowing a little more about their guidelines. Embark on a transformative journey through our Cyber security Bootcamp, where you'll delve deep into the intricacies of cutting-edge technologies like the SHA-256 algorithm. Uncover the cryptographic principles that make this algorithm the cornerstone of blockchain security, all while honing your skills in defending against cyber threats.

What are the Characteristics of the SHA-256 Algorithm?

Some of the standout features of the SHA algorithm are as follows:

- **Message Length:** The length of the cleartext should be less than 264 bits. The size needs to be in the comparison area to keep the digest as random as possible.
- **Digest Length:** The length of the hash digest should be 256 bits in SHA 256 algorithm, 512 bits in SHA-512, and so on. Bigger digests usually suggest significantly more calculations at the cost of speed and space.
- **Irreversible:** By design, all hash functions such as the SHA 256 are irreversible. You should neither get a plaintext when you have the digest beforehand nor should the digest provide its original value when you pass it through the hash function again.

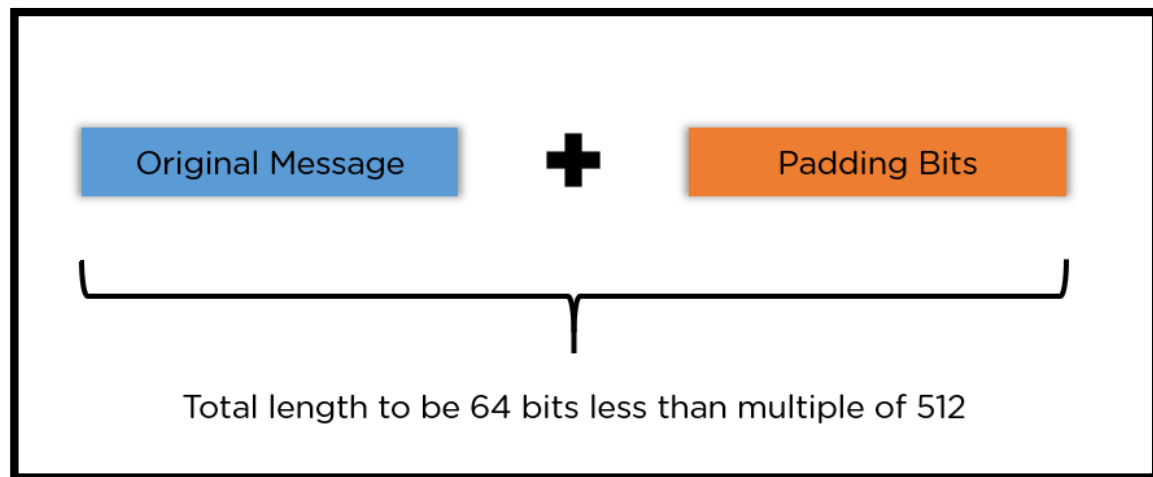
Now that you got a fair idea about the technical requirements for SHA, you can get into its complete procedure, in the next section.

Steps in SHA-256 Algorithm

You can divide the complete process into five different segments, as mentioned below:

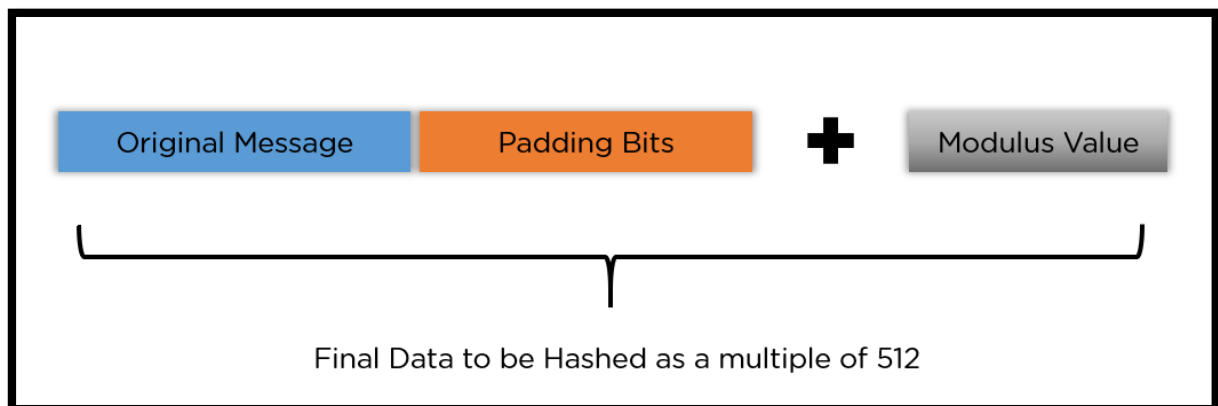
Padding Bits

It adds some extra bits to the message, such that the length is exactly 64 bits short of a multiple of 512. During the addition, the first bit should be one, and the rest of it should be filled with zeroes.

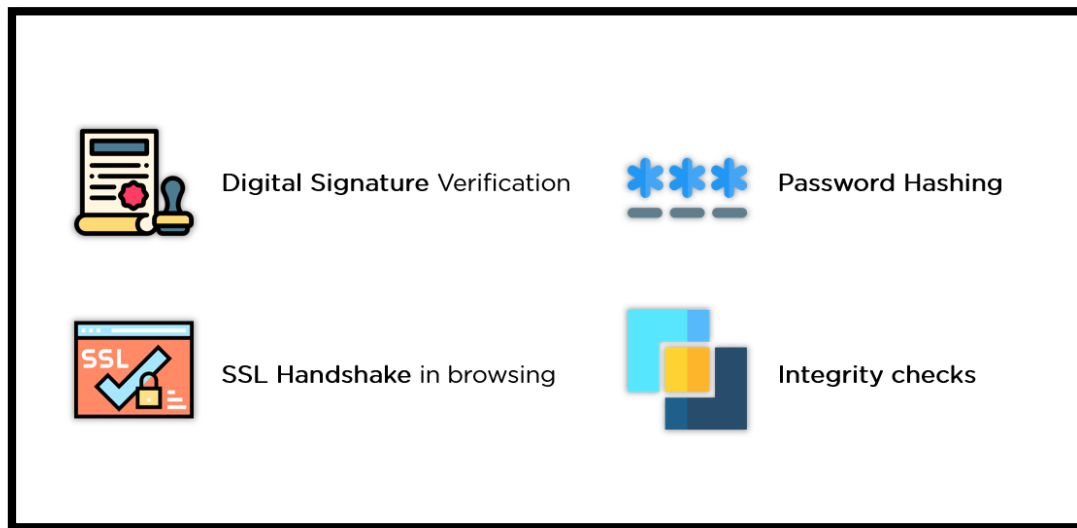


Padding Length

You can add 64 bits of data now to make the final plaintext a multiple of 512. You can calculate these 64 bits of characters by applying the modulus to your original cleartext without the padding.



Applications of SHA algorithm



As seen in the image above, the SHA algorithm is being used in a lot of places, some of which are as follows:

- **Digital Signature Verification:** Digital signatures follow asymmetric encryption methodology to verify the authenticity of a document/file. Hash algorithms like SHA 256 go a long way in ensuring the verification of the signature.
- **Password Hashing:** As discussed above, websites store user passwords in a hashed format for two benefits. It helps foster a sense of privacy, and it lessens the load on the central database since all the digests are of similar size.
- **SSL Handshake:** The SSL handshake is a crucial segment of the web browsing sessions, and it's done using SHA functions. It consists of your web browsers and the web servers agreeing on encryption keys and hashing authentication to prepare a secure connection.
- **Integrity Checks:** As discussed above, verifying file integrity has been using variants like SHA 256 algorithm and the MD5 algorithm. It helps maintain the full value functionality of files and makes sure they were not altered in transit.

WHAT IS SCRIPT?

Script is a password-based key derivation function that is particularly popular in the cryptocurrency industry due to its specific design that makes

mining more memory-intensive, hence offering a level of protection against large-scale custom hardware attacks. In essence, Scrypt enhances security by prioritizing memory access speed over computational speed, making it a preferred choice for many digital currencies.

How Scrypt Works (Step-by-Step)

Scrypt is designed to be **memory-intensive**, meaning it requires a lot of RAM to compute hashes. This makes it harder for ASIC miners (which are optimized for raw **computational** power but have limited memory bandwidth) to outperform general-purpose hardware.

Key Steps in Scrypt Hashing

1. Input Preparation

- Takes an input (e.g., block header + nonce).
- Uses a **salt** (random data) to prevent precomputation attacks (like rainbow tables).

2. Memory-Intensive Processing (PBKDF2 + Large RAM Usage)

- Fills a large buffer with pseudorandom data generated from repeated hashing (using HMAC-SHA256).
- This buffer is then accessed in a **pseudo-random order**, forcing miners to store it in RAM (not just compute quickly).

3. Final Hash Computation

- The data is mixed and compressed to produce the final hash.

Advantages OF Scrypt Over SHA-256

1. **ASIC Resistance:** Harder to optimize with specialized hardware.
2. **Fairer Mining:** Favors CPUs/GPUs over ASICs (though ASICs now exist for Scrypt).
3. **Security:** Protects against brute-force attacks due to memory demands.

Disadvantages OF Scrypt Over SHA-256

1. **Slower Verification:** Takes longer than SHA-256 for full nodes.
2. **Still Vulnerable to ASICs:** Modern Scrypt ASICs (e.g., for Litecoin) now exist.

Script vs. SHA-256 vs. Other Algorithms

Algorithm	Type	Memory Usage	ASIC Resistance	Used in
SHA-256	Cryptographic Hash	Low	(ASIC-dominated)	Bitcoin (BTC)
Script	Memory-Hard KDF	High	(Initially yes, now ASICs exist)	Litecoin (LTC)
Ethash	Memory-Hard PoW	High	(GPU-friendly)	Ethereum (pre-merge)
X11	Chained Hashes	Medium	(ASICs exist)	Dash (DASH)
RandomX	CPU-Optimized	Very High	(ASIC-resistant)	Monero (XMR)

X11 ALGORITHM

X11 is a chained hashing algorithm used in cryptocurrencies like Dash (DASH). It combines 11 different cryptographic hash functions to improve security, reduce energy consumption, and resist ASIC dominance (at least initially).

1. What is X11?

- **Type:** Proof-of-Work (PoW) hashing algorithm.
- **Key Feature:** Uses 11 different hash functions sequentially.
- **Created by:** Evan Duffield (2014) for Dash (originally Darkcoin).
- **Goal:**
 - Reduce energy consumption vs. Bitcoin's SHA-256.
 - Initially resist ASIC mining (though ASICs now exist).
 - Improve security through multi-layered hashing.

2. How X11 Works

X11 applies **11 different hash functions in sequence:**

1. **BLAKE**
2. **BMW (Blue Midnight Wish)**

3. **Grøstl**
4. **JH**
5. **Keccak** (precursor to SHA-3)
6. **Skein**
7. **Luffa**
8. **CubeHash**
9. **SHAvite-3**
10. **SIMD**
11. **ECHO**

Step-by-Step Process

1. Input Preparation

- The miner takes the **block header + nonce** as input.

2. First Hash (BLAKE)

- Input is hashed using **BLAKE**.

3. Second Hash (BMW)

- The BLAKE output is fed into **BMW**.

4. Continues Through All 11 Hashes

- The result passes through each function sequentially.

5. Final Hash Check

- If the output meets the **network difficulty**, the block is mined.
- If not, the miner changes the **nonce** and repeats.

Why Was X11 Created?

Advantages Over SHA-256 & Scrypt

1. **Lower Energy Use:** More efficient than Bitcoin's SHA-256.
2. **ASIC Resistance (Initially):** Harder to optimize than single-hash algorithms.
3. **Security through Diversity:** Breaking X11 requires cracking all 11 functions.

Disadvantages

1. **ASICs Now Exist:** Modern X11 ASIC miners (e.g., Baikal, iBelink) dominate.
2. **Complexity:** Harder to audit than single-hash algorithms.

X11 vs. Other Mining Algorithms

Algorithm	Hash Functions	ASIC Resistance	Energy Use	Used In
X11	11 chained	(Now ASIC-dominated)	Low	Dash
SHA-256	1 (SHA-256)	(ASIC-dominated)	High	Bitcoin
Scrypt	1 (Memory-hard)	(ASICs exist)	Medium	Litecoin
Ethash	DAG-based	(GPU-friendly)	Medium	Ethereum (pre-merge)
RandomX	CPU-optimized	(ASIC-resistant)	Low	Monero

-----QUESTIONS ON UNIT 4-----

- 1) Define Hill Climbing as a search algorithm. Explain its working principle with an example.
- 2) Differentiate between Hill Climbing and other local search algorithms like Simulated Annealing.
- 3) What are the key characteristics of a Hill Climbing algorithm? Explain with a real-world application.
- 4) What is the objective function in Hill Climbing? How does it influence the search process?
- 5) Explain the concept of "local maxima" in Hill Climbing and how it affects optimization.
- 6) Compare Simple Hill Climbing and Steepest Ascent Hill Climbing. Which is more efficient and why?
- 7) Explain the working of Steepest Ascent Hill Climbing with a suitable example.
- 8) What is Stochastic Hill Climbing? How does it differ from traditional Hill Climbing?
- 9) Discuss the advantages of Stochastic Hill Climbing over deterministic approaches.
- 10) Can Hill Climbing algorithms be modified for global optimization? Justify your answer.
- 11) List and explain three major disadvantages of Hill Climbing algorithms.
- 12) What are "foothills" in Hill Climbing? How do they impact the search process?
- 13) Define "plateaus" in Hill Climbing. Suggest a strategy to overcome them.
- 14) What are "ridges" in Hill Climbing? Why do they cause problems for the algorithm?
- 15) How does the problem of local optima affect Hill Climbing? Suggest two remedies.
- 16) How can random restarts improve Hill Climbing? Explain with an example.
- 17) Discuss how Simulated Annealing differs from Hill Climbing in escaping local optima.
- 18) Explain how Genetic Algorithms can be seen as an extension of Hill Climbing.
- 19) What is the role of heuristic functions in Hill Climbing? Provide an example.
- 20) Can Hill Climbing be used in real-time AI systems? Discuss its feasibility with pros and cons.

- 21) Define cryptocurrency. Explain its key features and how it differs from traditional fiat currency.
- 22) What is Bitcoin? Discuss its origin, purpose, and significance in the financial world.
- 23) Explain the concept of decentralization in cryptocurrencies. How does Bitcoin achieve it?
- 24) What is blockchain technology? How does it ensure security and transparency in Bitcoin transactions?
- 25) Discuss the role of miners in the Bitcoin network. How do they contribute to transaction validation?
- 26) List and explain five major real-world applications of cryptocurrencies.
- 27) How is Bitcoin used for cross-border transactions? Compare its efficiency with traditional banking systems.
- 28) Discuss the role of Bitcoin as a "store of value" (digital gold). What factors influence its price?
- 29) What are the risks and challenges of using cryptocurrencies for everyday payments?
- 30) What is SHA-256? Explain its role in Bitcoin mining and transaction security.
- 31) How does the Proof-of-Work (PoW) consensus mechanism in Bitcoin use SHA-256?
- 32) Compare SHA-256 with Scrypt in terms of computational efficiency and ASIC resistance.
- 33) Why is SHA-256 considered secure? Discuss its cryptographic properties.
- 34) What would happen if SHA-256 was compromised? How would it affect Bitcoin?
- 35) What is Scrypt? Why was it introduced as an alternative to SHA-256?
- 36) Explain how Scrypt's memory-hard function improves security and decentralization.
- 37) Which cryptocurrencies use Scrypt? Compare their mining efficiency with Bitcoin.
- 38) Discuss the advantages and disadvantages of Scrypt in cryptocurrency mining.
- 39) Can Scrypt resist ASIC miners? Justify your answer with real-world examples.
- 40) What is the X11 algorithm? How does its multi-hash approach improve security?
- 41) Explain how Dash (Digital Cash) utilizes the X11 algorithm for mining.